Q1. E-commerce company — 10 million product records: file organization strategy with cost analysis

Recommended strategy: Hashed file organization on Product_ID, supplemented by a B+ tree secondary index on frequently filtered attributes (category, price).

An e-commerce catalog is dominated by exact-match lookups (a customer clicking a specific product, or the system fetching a product by its ID during checkout/cart operations). Hashing gives O(1) average-case access for these point lookups, which is exactly the dominant workload.

Justification:

- Point lookups by Product_ID (add-to-cart, product page, inventory check) are O(1) with hashing vs O(log n) with a B+ tree and O(n) with a heap file.
- Range/filter queries (browse by category, price range, sort by rating) are handled by separate secondary B+ tree indexes rather than the primary organization, since hashing does not support range scans.
- Extendible or linear hashing is preferred over static hashing so the bucket directory can grow gracefully as the catalog scales without a full re-hash.

Cost analysis (10,000,000 records, assume 100-byte record, 4 KB block ⇒ ~40 records/block ⇒ ~250,000 blocks):

Organization	Search (by ID)	Insert	Delete	Disk I/O for point lookup
Heap file	O(n) — avg 125,000 blocks scanned	O(1) — append	O(n) to locate + remove	Very high (unacceptable at 10M scale)
Sorted file	O(log n) — ~18 I/Os (binary search)	O(n) — shift records	O(n) — shift records	Moderate, but insert/delete cost is prohibitive
Hashed file (recommended)	O(1) avg — 1–2 I/Os	O(1) avg	O(1) avg	Lowest — ideal for OLTP-style catalog access

Q2. Banking system — fast retrieval by account number and date range: indexing strategy

Recommended strategy: a Primary (clustering) index on Account_Number using a B+ tree, plus a Secondary (non-clustering) B+ tree index on Transaction_Date.

- Primary index — B+ tree on Account_Number: since transactions are typically stored/clustered physically by account, a dense/clustering B+ tree index gives fast exact-match retrieval (all transactions of one account) and keeps sequential range access efficient for statement generation.
- Secondary index — B+ tree on Transaction_Date: supports date-range queries (e.g., 'all transactions between 1 Jan and 31 Mar') independent of account. B+ trees are chosen over hashing here because the query is range-based, not point-based.
- Composite index option — a B+ tree on (Account_Number, Transaction_Date) is ideal when both filters are typically applied together (most common real banking query), because it satisfies the query using a single index traversal and returns results already sorted by date within an account.

Why B+ Tree over B-Tree/Hashing:

- B+ trees keep all data pointers in leaf nodes linked sequentially, making range scans (date ranges, statement periods) fast without extra lookups.
- Hash indexes are rejected for the date attribute since hashing cannot answer range predicates.
- Multi-level indexing keeps tree height small (typically 3–4 levels for millions of transactions), so any transaction is retrieved in 3–4 disk I/Os.

Q3. Healthcare system — records accessed by PatientID and Doctor Name: multi-level indexing solution

A two-tier multi-level indexing design is proposed:

1) Primary multi-level index on PatientID:

- PatientID is the primary/clustering key; records are stored physically sorted by PatientID.
- A multi-level B+ tree is built on top: leaf level indexes blocks of the data file, and successive levels index the level below, keeping the top few levels cached in memory.
- This gives O(logB N) search cost — for very large patient files this collapses to just 3–4 I/Os even with millions of records.

2) Secondary multi-level index on Doctor Name:

- Doctor Name is not the physical sort key, so a secondary (non-clustering) B+ tree index is built, where leaf entries store pointers (record IDs) to the actual patient records rather than the records themselves.
- Because many patients can share a doctor, this is implemented as a clustered bucket of record pointers per doctor (an inverted-list style secondary index) to avoid duplicate leaf entries and speed up 'all patients of Dr. X' queries.

Why multi-level over single-level:

A single-level index on a table of, say, 2 million patients would itself need ~2 million entries — too large to hold in one block. Multi-level indexing recursively indexes the index, reducing the number of blocks read per lookup from O(n) to O(logB n), where B is the fan-out per index block.

Q4. Logistics company — dynamic insertions/deletions of shipment records: B-Tree vs B+ Tree

Recommendation: B+ Tree.

Criterion	B-Tree	B+ Tree
Data storage	Data pointers in both internal and leaf nodes	Data pointers only in leaf nodes
Leaf linkage	Leaves not linked	Leaves linked as a sequential list
Insert/Delete rebalancing	Rebalancing can propagate through internal nodes holding data — more complex splits/merges	Rebalancing is confined to leaf level structure and simpler to propagate upward since internal nodes hold only keys
Range queries	Requires in-order traversal across internal nodes	Fast — simply walk the linked leaf list
Fan-out per node	Lower (data + keys share node space)	Higher (keys only) → shorter, flatter tree

For a logistics system, shipment records are constantly created (new shipments), updated (status changes), and deleted (completed/cancelled shipments) — a high-churn, dynamic workload. B+ Trees are more suitable because:

- Higher fan-out (no data payload in internal nodes) keeps the tree shallower, so each insert/delete touches fewer levels/blocks.
- Linked leaf nodes make it cheap to scan shipments by date, region, or status range — common logistics queries.
- Simpler, more localized rebalancing on split/merge since internal nodes carry only routing keys, reducing I/O cost per update.
- B+ Trees are the de-facto standard in production DBMS/file systems (MySQL InnoDB, PostgreSQL) precisely for this kind of dynamic, range-query-heavy workload.

Q5. HR system — 5000 employees: hashing scheme; static vs extendible hashing

Proposed hashing scheme:

- Hash key: Employee_ID.
- Hash function: h(Employee_ID) = Employee_ID mod N, where N is the number of buckets.
- Assume 20 records/bucket (4 KB block, ~200-byte employee record ⇒ 20 records/block) and a target load factor of 0.8 for headroom: buckets needed = 5000 / (20 × 0.8) ≈ 313, rounded to N = 320 buckets.

Static Hashing evaluation:

- Pros: simple, O(1) average lookup, no directory overhead — fine since 5000 employees is a small, fairly stable dataset (HR headcount grows slowly).
- Cons: if headcount grows well beyond the provisioned 320 buckets, bucket overflow chains form and performance degrades toward O(n) within a bucket; a full re-hash is needed to resize.

Extendible Hashing evaluation:

- Pros: uses a directory of pointers with a global depth; buckets split locally on overflow and the directory doubles only when needed — no full-file re-hash, graceful growth.
- Cons: extra indirection (directory lookup before bucket access) and directory-doubling overhead adds slight complexity/memory cost, which is unnecessary if the dataset is small and stable.

Recommendation: Static hashing is sufficient and more efficient for this scenario.

5000 employees is a small, low-growth-rate dataset typical of a single organization's HR system. The overhead of extendible hashing's directory indirection is not justified. Static hashing with ~20% extra bucket capacity (320 buckets for 5000 employees) comfortably absorbs normal headcount growth while keeping lookups at O(1) with minimal overflow. Extendible hashing would be preferred only if the company expected rapid, unpredictable growth (e.g., a fast-scaling startup or M&A-driven headcount surges).

Q6. Airline reservation system — point queries by flight number, range queries by date: combined index structure

Recommended structure: a two-index combination — Hash index on Flight_Number + B+ Tree index on Travel_Date, unified via a composite B+ Tree index on (Flight_Number, Travel_Date) for the common combined query.

- Hash index on Flight_Number: flight-number lookups ('show me flight AI-202') are exact-match, so hashing gives O(1) average access.
- B+ Tree index on Travel_Date: date-range queries ('flights available between 10 and 20 Dec') need ordered, sequential access, which only a tree-based index provides.
- Composite B+ Tree index on (Flight_Number, Travel_Date): since the most frequent real query combines both — 'flight AI-202 on dates between X and Y' — a single composite index avoids intersecting two separate index results and directly returns a sorted range for that flight.

This hybrid design lets each query type use the structure best suited to it: hashing for pure equality lookups, B+ tree ordering for pure range scans, and the composite index for the common combined case, minimizing overall I/O across the mixed workload.

Q7. University student database — file organization and secondary indexing for department and CGPA range queries

File organization: Sorted (clustered) file organization on Department, since department-wise queries and reports are the dominant access pattern (class lists, department rosters), and department has relatively few distinct, stable values.

Indexing design:

- Primary clustering index on Department: records are physically grouped/sorted by department, so 'list all students in Computer Science' is a fast sequential block scan.

- Secondary B+ tree index on CGPA: CGPA is not the physical sort key and range queries are needed ('students with CGPA > 8.5'), so a non-clustering B+ tree secondary index with leaf pointers to actual student records is built on CGPA.
- Optional composite secondary index on (Department, CGPA) to directly serve the common combined query 'top CGPA students within a department' without a separate join/intersection step.

This combination gives fast department-scoped scans (via clustering) and efficient CGPA range filtering (via the B+ tree secondary index), covering both stated access patterns with minimal storage and index-maintenance overhead.

Q8. Disk block utilization: Heap, Sorted, and Hashed file organizations — 100,000 records × 50 bytes, 4 KB blocks

Given:

- Record size = 50 bytes; Block size = 4096 bytes (4 KB); Total records = 100,000
- Records per block (packed) = ⌊4096 / 50⌋ = 81 records/block, using 81 × 50 = 4050 bytes (46 bytes unused per block)

Organization	Blocking factor used	Blocks required	Block utilization	Reasoning
Heap file	81 (fully packed, no order maintained)	⌈100,000 / 81⌉ = 1,235 blocks	≈ 98.9% (4050/4096 per block)	Records appended with no gaps — near-maximal packing since no free space is reserved for ordering.
Sorted file	81 initially, degrading over time	1,235 blocks initially; grows with overflow as inserts require order-preserving shifts	≈ 98.9% right after (re)organization, drops as scattered inserts create partially-filled overflow blocks	Sorted files are packed tightly at load time, but maintaining sort order under insertions forces periodic reorganization to restore utilization.

Organization	Blocking factor used	Blocks required	Block utilization	Reasoning
Hashed file	≈ 65 (bucket load factor kept at ~80% to limit overflow chains)	≈ 100,000 / (81 × 0.8) ≈ 1,543 blocks	≈ 79–80% (by design)	Buckets are deliberately under-filled to reserve room for future insertions and reduce overflow-chain length, trading space for stable O(1) access.

Interpretation: Heap and freshly-organized Sorted files achieve the highest space efficiency (~99%) because no capacity is reserved. Hashed files intentionally sacrifice ~20% utilization for insertion headroom, trading disk space for consistently fast O(1) access — an acceptable trade-off given disk is cheap relative to query latency in most systems.

Q9. Social media platform — 500 million posts indexed by user_id, timestamp, and hashtag: composite indexing strategy

Recommended strategy: a combination of a clustering hash/range-partitioned index on user_id, a B+ tree (or LSM-tree) index on timestamp, and an inverted index on hashtag — unified through partitioning and composite indexes for common query patterns.

- Index on user_id (equality-heavy: 'show this user's posts'): partition/shard the 500M posts by hash of user_id, and within each partition maintain a B+ tree or hash index for O(1)–O(log n) retrieval of a user's own posts.
- Index on timestamp (range-heavy: 'posts in the last 24 hours', chronological feeds): a B+ tree — or in write-heavy systems an LSM-tree (as used by Cassandra/RocksDB-backed platforms) — since posts are append-mostly and LSM trees optimize for high insert throughput while still supporting efficient range scans.
- Index on hashtag (multi-valued, search-heavy: 'all posts with #cricket'): an inverted index, where each hashtag maps to a sorted/compressed list (posting list) of post IDs — the standard structure for text/tag search at this scale, similar to search-engine indexing.
- Composite index on (user_id, timestamp): serves the very common 'this user's posts, most recent first' query directly, avoiding a separate sort step.

Why not a single index:

No single index type efficiently serves all three access patterns — equality on user_id, range on timestamp, and multi-valued search on hashtag have fundamentally different optimal structures. At 500M-row scale, combining hashing/B+ trees for structured lookups with an inverted index for hashtag search — all layered over horizontal partitioning (sharding by user_id or time) — is the standard, scalable design used by real social platforms.

Q10. Supermarket billing system — 1 million daily transactions: performance comparison of Heap, Sorted, and Hashed files

Operation	Heap File	Sorted File	Hashed File
Search (by key)	O(n) — average n/2 block scans; unacceptable at 1M/day	O(log n) — binary search, ~20 I/Os	O(1) average — 1–2 I/Os, best for point lookups
Insert	O(1) — append at end of file; fastest insert	O(n) — must shift records to preserve order	O(1) average — direct bucket placement (occasional overflow chain)
Delete	O(n) to find + O(1) to mark/remove	O(n) — find (log n) + shift remaining records	O(1) average to find + remove within bucket
Best suited for	Write-heavy logs (e.g., raw transaction log/audit trail)	Reporting/range queries (e.g., end-of-day sorted sales report)	Live billing lookups (e.g., 'find this transaction/receipt now')

Recommendation for the supermarket billing system:

Use a Hashed file organization on Transaction_ID for the live billing/POS layer, since search and insert dominate during store hours and both are O(1) average with hashing — critical at 1 million transactions/day (≈ 12 transactions/second sustained, with peak bursts). Retain a separate append-only Heap file as the raw transaction log for auditing (fastest possible inserts, no search needed). For end-of-day/periodic reporting (sales by time range, top products), batch-load the day's transactions into a Sorted file or a B+ tree index, where the one-time O(n log n) sort cost is amortized across many range-based reporting queries.